SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

Number Representation and Conversions

FY BTECH
Digital Logic Design

Number System

- Number system is a basis for counting varies items.
- Modern computers communicate and operate with binary numbers which use only the digits 0 &1.
- Basic number system used by humans is Decimal number system.

Number System

- Any Number can be defined as a collection of digits. E.g., 123.10 signifies a number with integer part 123 and fractional part 10 separated by a radix point (.)
- Some other commonly used number systems are

I. Binary number system (base 2)

II. Octal number system (base 8)

III. Hexadecimal Number system(base 16)

Number System

Table 2.1 ***Characteristics of Commonly Used Number Systems***

Number system	Base or radix (b)	Symbols used (d_i or d_{-f})	Weight assigned to position i	Weight assigned to position $-f$	Example
Binary	2	0, 1	2^i	2^{-f}	1011.11
Octal	8	0, 1, 2, 3, 4, 5, 6, 7	8^i	8^{-f}	3567.25
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	10^i	10^{-f}	3974.57
Hexadecimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, *A*, *B*, *C*, *D*, *E*, *F*	16^i	16^{-f}	3*FA*9.56

Figure 1 : Number System characteristics

Source: pg 29. Modern Digital electronics R.P. Jain

Binary Number system

- The binary number has a radix of 2.
- As r = 2, only two digits are needed, and these are 0 and 1.
- In binary system weight is expressed as power of 2.
- The left most bit, which has the greatest weight is called the Most Significant Bit (MSB).
- And the right most bit which has the least weight is called Least Significant Bit (LSB).

Octal Number System

- Digital systems operate only on binary numbers.
- Since binary numbers are often very long, two shorthand notations, octal and hexadecimal, are used for representing large binary numbers.
- Octal systems use a base or radix of 8.
- It uses first eight digits of decimal number system. Thus it has digits from 0 to 7.

Hexadecimal Number System

- It is very popular in computer systems
- Has base 16 so 16 distinct symbols are required for representation
- These are numerals 0 to 9 and alphabets from A to F
- Hence, alphanumeric number system

Hexadecimal Number System

Decimal	Binary	Hexa-Decimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Figure 2: Hexadecimal Number system

Source: https://media.geeksforgeeks.org/wp-content/uploads/20251217164605653284/Hexadecimal_number_system.webp

Decimal to Hexa-Decimal Conversion

Convert the following decimal numbers into hexadecimal numbers.

(a) 95.5 (b) 675.625

(a) *Integer part*

	Quotient	Remainder
95/16	5	15
5/16	0	5

5 F

Thus, $(95)_{10} = (5F)_{16}$

Fractional part

$$\begin{array}{r} 0.5 \\ \times\ 16 \\ \hline 8.0 \\ \downarrow \\ 8 \end{array}$$

Thus, $(0.5)_{10} = (0.8)_{16}$
Therefore, $(95.5)_{10} = (5F.8)_{16}$

Source: Modern Digital electronics R.P. Jain

Decimal to Hexa-Decimal Conversion

(b) *Integer part*

	Quotient	Remainder
675/16	42	3
42/16	2	10
2/16	0	2

2 A 3

Thus, $(675)_{10} = (2A3)_{16}$

Fractional part

$$\begin{array}{r} 0.625 \\ \times 16 \\ \hline 10.000 \end{array}$$

$\downarrow$

A

Thus, $(0.625)_{10} = (0.A)_{16}$

Therefore, $(675.625)_{10} = (2A3.A)_{16}$

Source: Modern Digital electronics R.P. Jain

Hexa-Decimal to Decimal Conversion

Example 2.28

Obtain decimal equivalent of hexadecimal number $(3A.2F)_{16}$

Solution

Using Eq. (2.1),

$$\begin{aligned}(3A.2F)_{16} &= 3 \times 16^1 + 10 \times 16^0 + 2 \times 16^{-1} + 15 \times 16^{-2} \\ &= 48 + 10 + \frac{2}{16} + \frac{15}{16^2} \\ &= (58.1836)_{10}\end{aligned}$$

Source: pg 49. Modern Digital electronics R.P. Jain

Hexa-Decimal to Binary

Example 2.30

Convert $(2F9A)_{16}$ to equivalent binary number.

Solution

Using Table 2.7, find the binary equivalent of each hex digit.

$$(2F9A)_{16} = (0010\ 1111\ 1001\ 1010)_2$$
$$= (0010111110011010)_2$$

Source: Modern Digital electronics R.P. Jain

Binary to Hexa-decimal

Example 2.31

Convert the following binary numbers to their equivalent hex numbers.

(a) 10100110101111
(b) 0.00011110101101

Solution

(a) $$(10100110101111)_2 = \underbrace{0010}_{2}\ \underbrace{1001}_{9}\ \underbrace{1010}_{A}\ \underbrace{1111}_{F}$$

$$\therefore (10100110101111)_2 = (29AF)_{16}$$

(b) $$(0.00011110101101)_2 = \underbrace{0.0001}_{1}\ \underbrace{1110}_{E}\ \underbrace{1011}_{B}\ \underbrace{0100}_{4}$$

$$\therefore (0.00011110101101)_2 = (0.1EB4)_{16}$$

Source: Modern Digital electronics R.P. Jain

Hex-to-Octal Conversion

Example 2.33

Convert the following hex numbers to octal numbers.

(a) $A72E$ (b) $0.BF85$

Solution

(a)
$$(A72E)_{16} = (1010\ 0111\ 0010\ 1110)_2$$
$$= \underbrace{001}\ \underbrace{010}\ \underbrace{011}\ \underbrace{100}\ \underbrace{101}\ \underbrace{110}$$
$$= (123456)_8$$

(b)
$$(0.BF85)_{16} = (0.1011\ 1111\ 1000\ 0101)_2$$
$$= 0.\underbrace{101}\ \underbrace{111}\ \underbrace{111}\ \underbrace{000}\ \underbrace{010}\ \underbrace{100}$$
$$= (0.577024)_8$$

Source: Modern Digital electronics R.P. Jain

SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Signed Binary Numbers

- In decimal number system ‘+’ sign indicates positive numbers and ‘-’ sign indicates negative number.
- The positive sign is dropped and the absence of sign indicates positive values
- This representation is known as signed numbers
- The binary number system uses only two digits, 0 and 1, to represent all data in computing and digital electronics
- It is necessary to represent the sign magnitude using the same digits as well
- **Left most MSB ‘0’** is used to denote **positive number**
- **Left most MSB ‘1’** is used to denote **negative number**

Range of Signed and Unsigned Numbers

- **Range of Unsigned Numbers in Binary System:**
  - The range depends directly on the bit length. For an n-bit number:
  - Minimum value: 0
  - Maximum value: $2^n - 1$
- **Range of Signed Numbers in Binary System:**
  - Minimum value: $-2^{n-1} - 1$
  - Maximum value: $2^{n-1} - 1$

Signed Binary Numbers (cont.)

Find the decimal equivalent of the following binary numbers assuming sign-magnitude representation of the binary numbers.

(a) 101100 (b) 001000

Solution

(a) Sign bit is 1, which means the number is negative.

$$\text{Magnitude} = 01100 = (12)_{10}$$

$$\therefore \quad (101100)_2 = (-12)_{10}$$

(b) Sign bit is 0, which means the number is positive.

$$\text{Magnitude} = 01000 = 8$$

$$\therefore \quad (001000)_2 = (+8)_{10}$$

One's Complement form

- In binary number, if each 1 is replaced by 0 and 0 is replaced by 1 the resulting number is called ones complement
- Both numbers complement each other is one is positive the other is negative
- In one's complement if the **MSB** is 'zero' it represents a positive number and 'one' represents negative number
- Eg. $(\mathbf{0}101)_2 = (+5)_{10}$ and $(\mathbf{1}101)_2 = (-5)_{10}$

One's Complement form Numerical

Example 2.8

Find the one's complement of the following binary numbers.

(a) 0100111001 (b) 11011010

Solution

(a) 1011000110 (b) 00100101

Example 2.9

Represent the following numbers in one's complement form.

(a) +7 and −7 (b) +8 and −8 (c) +15 and −15

Solution

In one's complement representation,

(a) $(+7)_{10} = (0111)_2$ and $(-7)_{10} = (1000)_2$

(b) $(+8)_{10} = (01000)_2$ and $(-8)_{10} = (10111)_2$

(c) $(+15)_{10} = (01111)_2$ and $(-15)_{10} = (10000)_2$

Two's Complement Form

- If one is added to a ones complement you get an two's complement
- For example, 2's complement of 0101 is 1011. Since 0101 represents $(+5)_{10}$therefore, 1011 represents $(-5)_{10}$ in 2's complement representation
- If the MSB is 0 the number is positive, whereas if the MSB is 1 the number is negative
- For an n-bit number, the maximum positive number which can be represented in 2's complement form is $(2^{n-1} - 1)$ and the maximum negative number is (-2^{n-1})
- Two's complement of the two's complement of a number is the number itself

Two's Complement Form Numerical

Example 2.10

Find the 2's complement of the numbers:

(i) 01001110 (ii) 00110101

Solution

(i) Number 0 1 0 0 1 1 1 0
1's complement 1 0 1 1 0 0 0 1
Add 1 1

1 0 1 1 0 0 1 0

(ii) Number 0 0 1 1 0 1 0 1
1's complement 1 1 0 0 1 0 1 0
Add 1 1

1 1 0 0 1 0 1 1

Source: Modern Digital electronics R.P. Jain

Binary Addition

- The rules to be applied for binary addition are as follows:

Augend	Addend	Sum	Carry	Result
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	0	1	10

- 1+1+1 = Sum:1 Carry: 1

Binary Addition Numerical

Example 2.13

Add the binary numbers:

(i) 1011 and 1100 (ii) 0101 and 1111

Solution

(i)

		1	0	1	1
	(+)	1	1	0	0
	1	0	1	1	1
	↑ carry				

(ii)

		(1)	(1)	(1)		← carry
		0	1	0	1	
	(+)	1	1	1	1	
	1	0	1	0	0	
	↑ carry					

Binary Subtraction

- The rules to be applied for binary subtraction are as follows:

Minuend	Subtrahend	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Binary Subtraction Numerical

Example 2.15

Perform the following subtraction:

$$\begin{array}{r} 1\,0\,1\,1 \\ -\ 0\,1\,1\,0 \\ \hline \end{array}$$

Solution

	Column 4	3	2	1	
	1	0	1	1	Minuend
(–)	0	1	1	0	Subtrahend
	0	1	0	1	Difference

Here, in columns 1 and 2, borrow = 0 and in column 3 it is 1. Therefore, in column 4 first subtract 0 from 1 and from this result obtained subtract the borrow bit.

Subtraction using 2’s complement

- Binary subtraction can be performed by adding the 2’s complement of the subtrahend to the minuend.
- If a final carry is generated, discard the carry and the answer is given by the remaining bits which is positive (the minuend is greater than the subtrahend).
- If the final carry is 0, the answer is negative (the minuend is smaller than the subtrahend) and is in 2’s complement form.

Subtraction using 2's complement Numerical

Example 2.18

Perform binary subtraction using 2's complement representation of negative numbers.

(i)

$$\begin{array}{r} 7 \\ -5 \\ \hline +2 \end{array}$$

(i)

$$\begin{array}{r} 7 \\ -5 \\ \hline +2 \end{array} \Rightarrow \begin{array}{rrrrrl} & 0 & 1 & 1 & 1 & \text{Minuend} \\ (+) & 1 & 0 & 1 & 1 & \text{2's complement of subtrahend} \\ \hline 1 & 0 & 0 & 1 & 0 & \end{array}$$

$\uparrow$

Discard final carry

The answer is 0 0 1 0 equivalent to $(+2)_{10}$.

Subtraction using 2's complement Numerical cont..

Example 2.18

Perform binary subtraction using 2's complement representation of negative numbers.

(ii)

$$\begin{array}{r} 5 \\ -7 \\ \hline -2 \end{array}$$

(ii)

$$\begin{array}{r} 5 \\ -7 \\ \hline -2 \end{array} \Rightarrow \begin{array}{rl} 0\ 1\ 0\ 1 & \text{Minuend} \\ (+)\ 1\ 0\ 0\ 1 & \text{2's complement of subtrahend} \\ \hline 1\ 1\ 1\ 0 & \end{array}$$

The final carry = 0. Therefore, the answer is negative and is in 2's complement form. 2's complement of 1110 = 0010

Therefore, the answer is $(-2)_{10}$.

Binary Coded Decimal

- In this code, decimal digits 0 through 9 are represented (coded) by their natural binary equivalents using four bits and each decimal digit of a decimal number is represented by this four bit code individually
- For example, $(23)_{10}$ is represented by 0010 0011 using BCD code, rather than $(10111)_2$ From this it is observed that it requires more number of bits to code a decimal number using BCD code than using the straight binary code.
- However, in spite of this disadvantage it is very convenient and useful code for input and output operations in digital systems.
- This code is also known as 8-4-2-1 code or simply BCD code. 8, 4, 2, and 1 are the weights of the four bits of the binary code of each decimal digit similar to straight binary number system.
- Therefore, this is a weighted code and arithmetic operations can be performed using this code

Binary Coded Decimal Numerical

Represent the decimal number 27

Each digit of the decimal number is coded using 4-bit BCD code as given below

0010 0111

Gray Code

- Unit Distance Property: A non-weighted code where each number differs from its predecessor and successor by only a single bit.
- Applications: Used extensively for shaft encoders due to its single-bit change property.

Reflected Code Construction:

- **1-bit**: Consists of code words 0 and 1.
- **n-bit**: The first 2^{n-1} codes are (n-1)-bit codes with a leading 0 appended.
- **Reflection**: The remaining 2^{n-1} codes are the (n-1)-bit codes in reverse order (mirrored) with a leading 1 appended.

Gray Code

Figure: Shaft encoder

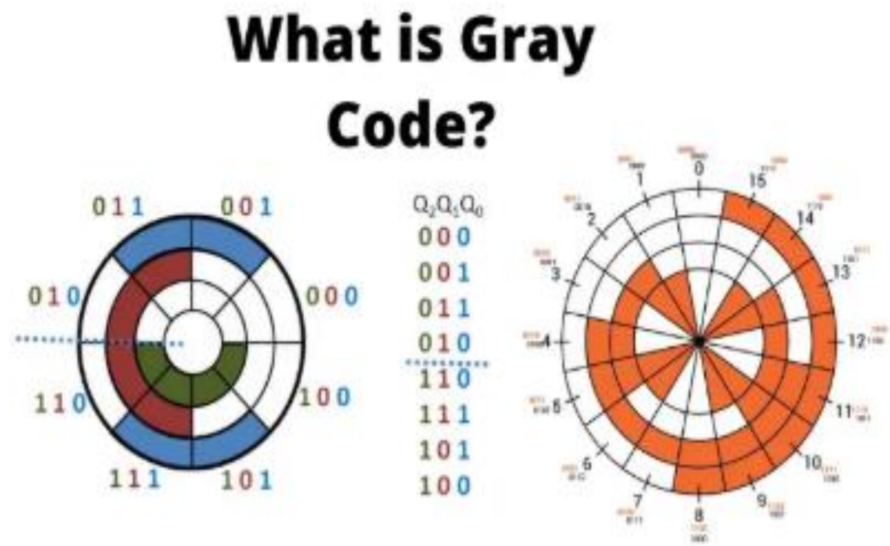


Figure: Gray code for Shaft encoder

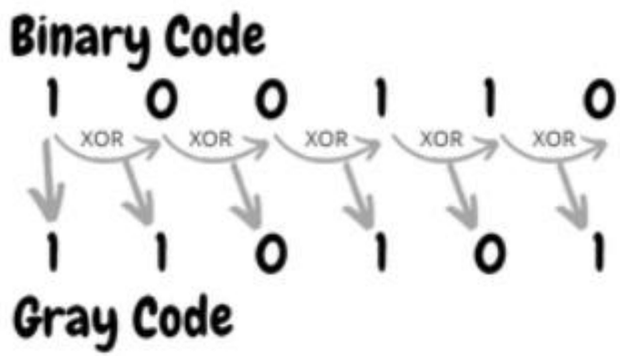
Gray Code

Decimal	Gray Code	Binary
0	0000	0000
1	0001	0001
2	0011	0010
3	0010	0011
4	0110	0100
5	0111	0101
6	0101	0110
7	0100	0111
8	1100	1000
9	1101	1001
10	1111	1010
11	1110	1011
12	1010	1100
13	1011	1101
14	1001	1110
15	1000	1111

https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcSk5jmQ-4u6Bxl87buZkT9J0vIJN_nAgnYt7Hk8PmQ27w&s

How to generate Gray Code from binary

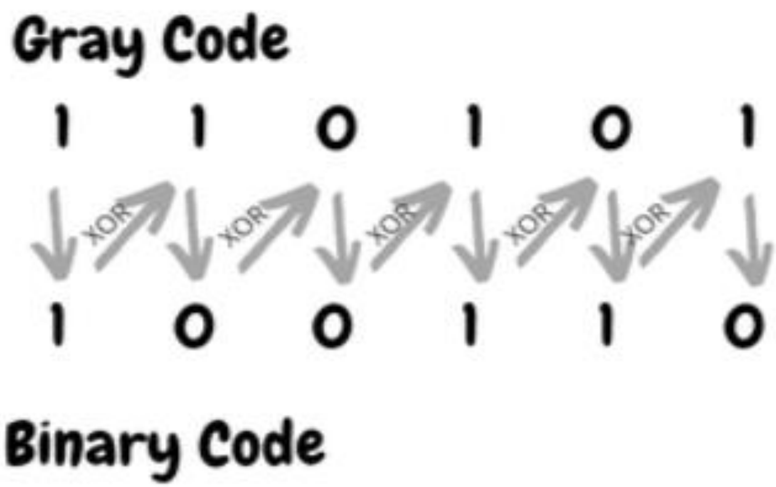
- The Most Significant Bit (MSB) of the gray code is always equal to the MSB of the given binary code.
- Other bits of the output gray code can be obtained by XORing binary code bit at that index and previous index.



Source: https://www.geeksforgeeks.org/dsa/gray-to-binary-and-binary-to-gray-conversion/

How to generate binary Code from Gray

- The Most Significant Bit (MSB) of the binary code is always equal to the MSB of the given gray code.
- Other bits of the output binary code can be obtained by checking the gray code bit at that index. If the current gray code bit is 0, then copy the previous binary code bit, else copy the invert of the previous binary code bit.



Concept of Parity for error

- In digital electronics, parity is an error-checking technique that uses an extra binary digit, called a parity bit, to ensure data integrity.
- It is added to a binary code so that the total number of '1's in the data is either always even (even parity) or always odd (odd parity).
- At the receiving end, the parity bit is checked to detect if any single-bit errors have occurred during transmission.

Parity for Error Detection

Parity generator:

- A parity generator adds a parity bit to a block of data.
- **Even parity**: A '0' is added if there is an even number of '1's, and a '1' is added if there is an odd number of '1's, to make the **total count even**.
- **Odd parity**: A '1' is added if there is an even number of '1's, and a '0' is added if there is an odd number of '1's, to make the **total count odd**.

Transmission: The data along with the parity bit is transmitted.

Parity checker:

- The receiver counts the number of '1's in the received data, including the parity bit.
- **Even parity**: If the count is even, the data is accepted as correct. If the count is odd, a parity error is detected.
- **Odd parity**: If the count is odd, the data is accepted. If the count is even, an error is detected.

Error detection:

- If the parity check fails, the data is flagged as corrupted. This method can only detect single-bit errors; if two bits flip, the parity will still appear correct.

Parity for Error Detection (cont..)

Parity generator:

- Given data : 1101
- Even parity: Parity Bit = 1 so that total no of '1's even
- Odd parity: Parity Bit = 0 so that total no of '1's odd.

Transmission: The data along with the parity bit is transmitted.

Parity checker:

- The receiver counts the number of '1's in the received data, including the parity bit.
- Even parity: Received Data : 11010 …. Count is odd there is error
- Odd parity: Received Data : 11010 ….Count is odd so data is correct